

CINEMATE: AI-POWERED MOVIE RECOMMENDATION CHATBOT

Submitted by

ASWIN SATHEESH

2116231801016

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



**RAJALAKSHMI ENGINEERING COLLEGE (AUTONOMOUS) THANDALAM
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
ANNA UNIVERSITY, CHENNAI 600 025**

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project titled CineMate: AI-Powered Movie Recommendation Chatbot using Google Gemini API and Serverless Architecture (Phase 2) is the bonafide work of ASWIN SATHEESH (2116231801016) who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. J M GNANASEKAR

Head of the Department

Department of Artificial Intelligence and
Data Science,

Rajalakshmi Engineering College Thandalam,
Chennai – 602105.

Mr. A SURENDAR

Assistant Professor

Department of Artificial Intelligence and
Data Science,

Rajalakshmi Engineering College Thandalam,
Chennai – 602105.

Certified that the candidate was examined in VIVA-VOCE Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project entitled CineMate: AI-Powered Movie Recommendation Chatbot using Google Gemini API and Serverless Architecture (Phase 2) is a bonafide work carried out by us under the supervision of Mr. Surendar A, Assistant Professor, Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College, Thandalam, Chennai. This work has not been submitted in full or in part for the award of any other degree or diploma in this or any other university.

ASWIN SATHEESH (231801016)

ACKNOWLEDGEMENT

We thank the Almighty for being with us through every walk of our life and showering blessings throughout this endeavour. We sincerely thank our respected Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., and our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., for providing us with the requisite infrastructure and for their sincere efforts in educating us in their premier institution.

We sincerely thank Dr. S. N. MURUGESAN, M.E., Ph.D., our beloved Principal, for his kind support and the facilities provided to complete this work in time. We express our sincere thanks to Dr. J M GNANASEKAR, Ph.D., Professor and Head of the Department of Artificial Intelligence and Data Science, for his guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, Mr. Surendar A, Assistant Professor, Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College, for his invaluable guidance, constant motivation, and thoughtful suggestions throughout the course of this project. His constructive feedback at every stage of development has been instrumental in shaping the quality of this work. We are grateful to all faculty members of the Department of Artificial Intelligence and Data Science for their support, encouragement, and insightful discussions. We also thank our fellow students and friends who provided moral support and inspiration during the development of this project.

ABSTRACT

The rapid proliferation of digital media content across global streaming platforms has amplified the need for intelligent, personalized, and conversationally natural recommendation systems. CineMate Phase 2 represents a significant architectural and functional evolution from its rule-based predecessor. This project presents a production-ready, AI-powered movie recommendation chatbot built upon Google's Gemini Flash (gemini-flash-latest) large language model, deployed on a serverless infrastructure using the Vercel platform.

Unlike the Phase 1 system, which relied on keyword-based intent detection and a static movie dataset, CineMate Phase 2 leverages the generative and reasoning capabilities of a state-of-the-art large language model to produce dynamic, contextually rich, and personalized movie recommendations. The system employs a carefully engineered prompt architecture that instructs the model to produce structured responses containing an overview, a curated list of movie recommendations, justifications for each recommendation, and representative cinematic examples.

The frontend, implemented using HTML5, CSS3, and JavaScript, delivers a ChatGPT-style conversational interface with support for multiple independent chat sessions, persistent session history via browser localStorage, a collapsible sidebar for session management, and a responsive dark-themed layout. Backend logic is encapsulated within a Vercel serverless function (api/chat.js) that securely relays requests to the Gemini API without exposing the API key in client-side code.

Conversational memory is maintained by transmitting the full message history with each API request, enabling the model to produce contextually coherent responses across multi-turn interactions. Experimental evaluations demonstrate high output relevance, accurate context retention across conversation turns, and average response latencies below two seconds under standard network conditions. CineMate Phase 2 establishes a robust foundation for further enhancements including voice interaction, multilingual support, and adaptive preference learning.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Need for Intelligent Recommendation Systems	2
1.3 Limitations of Rule-Based Systems (Phase 1).....	2
1.4 Problem Statement	3
1.5 Objectives	3
CHAPTER 2: LITERATURE SURVEY	5
2.1 Traditional Recommendation Systems.....	5
2.2 Rule-Based Chatbots vs AI Chatbots	5
2.3 LLM-Based Systems (Gemini and Modern AI Chatbots).....	6
2.4 Limitations of Existing Systems.....	7
CHAPTER 3: REQUIREMENTS.....	9
3.1 Hardware Requirements.....	9
3.2 Software Requirements	9
3.2.1 HTML5, CSS3, and JavaScript	10
3.2.2 Vercel Platform.....	10
3.2.3 Google Gemini API.....	10
3.2.4 GitHub	11
CHAPTER 4: SYSTEM ARCHITECTURE	12
4.1 Overall System Architecture.....	12
4.2 Architecture Flow	12
4.3 Module Description	13
4.3.1 UI Layer.....	13
4.3.2 API Layer (Serverless Function)	13
4.3.3 AI Layer (Gemini).....	14
4.3.4 Storage Layer (localStorage)	14
CHAPTER 5: METHODOLOGY	15
5.1 Input Processing.....	15
5.2 API Communication Flow.....	15
5.3 Prompt Engineering Strategy	16
5.4 Context Memory Handling.....	16
5.5 Response Structuring (Overview, Movies, Why, Examples)	17
CHAPTER 6: IMPLEMENTATION AND DEPLOYMENT.....	18
6.1 File Structure Explanation	18

6.2 Frontend Implementation	18
6.2.1 HTML Structure (index.html).....	18
6.2.2 CSS Design (style.css)	19
6.2.3 JavaScript Application Logic (script.js)	19
6.3 Serverless Function Implementation (api/chat.js)	19
6.4 Gemini API Integration using gemini-flash-latest	20
6.5 Deployment on Vercel	20
CHAPTER 7: EXPERIMENTAL RESULTS.....	22
7.1 Testing with Different Inputs	22
7.2 Output Accuracy and Relevance	23
7.3 Context Handling Verification	23
7.4 Performance Evaluation	24
CHAPTER 8: DISCUSSION	26
8.1 Comparison with Phase 1 Chatbot.....	26
8.2 Improvements Achieved	27
8.3 Observations.....	27
CHAPTER 9: ADVANTAGES AND LIMITATIONS	29
9.1 Advantages.....	29
9.1.1 AI-Powered Dynamic Responses	29
9.1.2 Personalized Recommendations	29
9.1.3 Scalable Serverless Architecture	29
9.1.4 Secure API Key Management.....	29
9.2 Limitations	30
9.2.1 Internet and API Dependency	30
9.2.2 Response Variability.....	30
9.2.3 API Usage Limits and Costs	30
CHAPTER 10: CONCLUSION AND FUTURE SCOPE	32
10.1 Conclusion	32
10.2 Future Scope.....	32
10.2.1 Voice-Based Interaction	32
10.2.2 Multi-Language Support	33
10.2.3 User Preference Learning	33
10.2.4 Retrieval-Augmented Generation	33
CHAPTER 11: REFERENCES	34

CHAPTER 1: INTRODUCTION

1.1 Background

The global entertainment industry has undergone a seismic transformation over the past two decades, driven primarily by the mass adoption of internet-connected streaming platforms. Services such as Netflix, Amazon Prime Video, Disney+, Apple TV+, and Hulu collectively curate catalogues that span tens of thousands of titles across genres, languages, and production eras. This unprecedented abundance of content, while advantageous in principle, introduces the well-documented cognitive phenomenon of choice overload. Research in consumer psychology consistently demonstrates that an excess of available options leads to decision fatigue, diminished satisfaction with the eventual selection, and increased probability of abandoning the decision-making process entirely without making a choice.

Recommendation systems were conceived and developed in direct response to this challenge. The earliest algorithmic approaches, emerging in the mid-1990s with systems such as GroupLens and Ringo, employed collaborative filtering — a technique that identifies users with similar preference histories and leverages those commonalities to generate personalized content suggestions. Subsequent research introduced content-based filtering, which analyses the intrinsic attributes of items — genre, director, cast, narrative themes, and critical reception — to identify candidates similar to those a user has previously engaged with positively. The industrial recommendation systems deployed by Netflix, Spotify, and YouTube employ sophisticated hybrid architectures that integrate collaborative filtering, content-based analysis, and deep neural networks trained on hundreds of millions of interaction records.

Parallel to the evolution of recommendation systems, conversational artificial intelligence has emerged as a transformative modality for human-computer interaction. Beginning with early rule-based systems such as ELIZA and progressing through the development of statistical dialogue managers and, most recently, large language model (LLM)-powered chatbots such as OpenAI's ChatGPT, Anthropic's Claude, and Google's Gemini, conversational AI has demonstrated remarkable capability in understanding natural language queries and generating contextually appropriate, informationally rich responses. The convergence of recommendation intelligence and conversational natural language interfaces represents a particularly compelling application domain, enabling users to discover content through dialogue rather than passive browse-and-filter interaction paradigms.

1.2 Need for Intelligent Recommendation Systems

The limitations of purely algorithmic recommendation systems become apparent when examined from the perspective of user intent at the moment of query. Standard collaborative filtering systems are fundamentally retrospective: they infer future preferences from historical behaviour, rendering them ineffective for users who lack substantial interaction histories, who are seeking recommendations outside their established taste profile, or who wish to discover content appropriate to a specific situational context rather than a generalized preference pattern.

Users frequently seek recommendations driven by immediate contextual factors that standard systems are ill-equipped to process: a current emotional state (seeking catharsis after a difficult day), a social viewing context (an appropriate film for a family gathering with children of various ages), an available time window (a compact ninety-minute film before bed), or a thematic curiosity sparked by a recent news event or cultural conversation. These nuanced, context-sensitive recommendation needs are precisely the scenarios in which a conversational AI interface excels — the user can articulate their need in natural language, and the system can reason over that intent to produce a tailored recommendation set.

Furthermore, the advent of powerful, accessible LLM APIs has democratized the development of AI-powered conversational applications. Developers can now integrate state-of-the-art language model capabilities into web applications with relatively modest engineering effort, enabling the construction of intelligent, dynamic recommendation interfaces that would previously have required substantial machine learning infrastructure and domain-specific model training. CineMate Phase 2 is designed to demonstrate and leverage this capability, offering a production-quality AI-powered recommendation experience delivered through a lightweight, serverless web architecture.

1.3 Limitations of Rule-Based Systems (Phase 1)

The Phase 1 implementation of CineMate demonstrated the feasibility of a conversational movie recommendation interface built entirely on client-side web technologies without external dependencies. However, an honest assessment of the Phase 1 system reveals several fundamental limitations that are intrinsic to its rule-based, keyword-driven architecture and that motivate the comprehensive re-architecture undertaken in Phase 2.

The most significant limitation of the Phase 1 system is its dependence on a finite, manually curated movie dataset. The system can only recommend films explicitly included in its JavaScript

dataset array; it cannot reason about films outside that set, cannot incorporate recent releases, and cannot provide recommendations informed by nuanced or domain-specific knowledge that was not encoded by the dataset author. The system is, in essence, bounded by the knowledge and effort invested in its dataset construction.

The Phase 1 intent detection engine, while functional for straightforward genre, mood, and context queries, exhibits brittle behaviour under input variation. Synonymous phrasings, compound intents, negations, and conversational ambiguities routinely confound the keyword-matching logic, producing either irrelevant recommendations or fallback responses. Additionally, the Phase 1 system cannot engage in genuinely open-ended conversation about cinema — it cannot discuss a film's themes, compare directorial styles, explain narrative structures, or answer follow-up questions beyond the predefined keyword vocabulary. These limitations collectively constrain the user experience to a rigid query-response pattern rather than a genuinely conversational interaction.

1.4 Problem Statement

Users seeking personalized and contextually appropriate movie recommendations through a conversational interface encounter systems that are either constrained by static, manually maintained datasets and brittle keyword-matching logic (as in rule-based systems), or that require cloud-dependent inference infrastructure with associated costs and privacy implications (as in large-scale recommendation engines). Neither category of existing system simultaneously satisfies the requirements of open-ended conversational intelligence, dynamic and comprehensive movie knowledge, secure and cost-effective API access, multi-session conversational memory, and production-ready deployment on accessible web infrastructure.

The problem addressed in this project is the design and implementation of CineMate Phase 2 — an AI-powered movie recommendation chatbot that integrates Google's Gemini Flash large language model via a secure serverless API function, delivers dynamic and contextually rich recommendations through a structured response format, maintains conversational context across multi-turn interactions, supports multiple persistent chat sessions via browser localStorage, and is deployed on Vercel's serverless platform for scalable, production-grade accessibility.

1.5 Objectives

The primary objectives of this project are enumerated below:

- To design and implement a production-ready AI-powered movie recommendation chatbot, CineMate Phase 2, that leverages Google's Gemini Flash language model to generate dynamic, contextually rich, and personalized recommendations in response to natural language user queries.
- To architect a secure serverless backend using Vercel Functions that proxies all Gemini API communication, ensuring that the API key is never exposed in client-side code and that the application conforms to security best practices for production web deployments.
- To engineer a prompt architecture that instructs the Gemini model to produce structured recommendation responses consistently organized into four sections: an Overview, a list of recommended Movies, Reasons for each recommendation, and cinematic Examples.
- To implement a conversational memory mechanism that transmits the full session message history with each API request, enabling the model to maintain context coherence across multi-turn interactions without server-side session storage.
- To build a ChatGPT-style conversational user interface using HTML5, CSS3, and JavaScript that supports multiple independent chat sessions, persistent session history via browser localStorage, a collapsible sidebar for session navigation, typing indicators, and a responsive dark-themed layout.
- To evaluate the system's performance across dimensions of recommendation relevance, context retention accuracy, and response latency, and to document the results systematically.
- To establish an extensible architectural foundation that facilitates future enhancements including voice interaction, multilingual support, and user preference learning.

CHAPTER 2: LITERATURE SURVEY

2.1 Traditional Recommendation Systems

The academic and industrial literature on automated recommendation systems spans three decades and encompasses a broad spectrum of algorithmic approaches. Collaborative filtering, introduced in the mid-1990s by Goldberg et al. with the Tapestry system and subsequently refined through the GroupLens project at the University of Minnesota, operates on the principle that users with similar historical preferences are likely to share preferences for unseen items. User-based collaborative filtering computes pairwise user similarity scores — typically using Pearson correlation or cosine similarity — and generates recommendations by aggregating the ratings of similar users on unrated items.

Item-based collaborative filtering, introduced at scale by Amazon's Linden, Smith, and York in their seminal 2003 paper, addresses the scalability limitations of user-based approaches by pre-computing item-to-item similarity matrices and generating recommendations at query time through lightweight lookups. This architecture, which powers Amazon's product recommendation engine, demonstrated that recommendation quality comparable to user-based methods could be achieved with substantially lower computational overhead at inference time. The MovieLens dataset, maintained by the GroupLens research group, has served as the standard benchmark for academic recommendation system evaluation for over two decades, enabling systematic comparison of algorithmic variants.

Matrix factorization methods, including Singular Value Decomposition (SVD) and its probabilistic variants such as Probabilistic Matrix Factorization (PMF) and Bayesian Probabilistic Matrix Factorization (BPMF), achieved state-of-the-art performance on the Netflix Prize benchmark and established model-based collaborative filtering as the dominant industrial paradigm. Deep learning approaches, including Autoencoders for Collaborative Filtering (AutoRec) and Neural Collaborative Filtering (NCF), subsequently demonstrated that non-linear interaction modelling through neural architectures could further improve recommendation accuracy. However, all of these approaches require substantial offline training infrastructure, large interaction datasets, and server-side inference pipelines — characteristics that render them unsuitable for lightweight, conversational, or client-side deployment scenarios.

2.2 Rule-Based Chatbots vs AI Chatbots

Conversational interfaces for content recommendation have evolved along two distinct trajectories: rule-based systems and machine learning-driven AI systems. Rule-based chatbots, exemplified by early systems such as ELIZA (Weizenbaum, 1966) and ALICE (Wallace, 2009), operate by matching user input against a predefined library of pattern templates and responding with associated response templates. The ELIZA system demonstrated that superficially convincing conversational behaviour could be produced through pattern matching alone, a finding that spurred decades of chatbot research. Rule-based systems excel within well-defined, constrained domains where the space of user intents is finite and can be exhaustively enumerated in the pattern library.

The introduction of statistical dialogue systems and, subsequently, neural sequence-to-sequence models represented a paradigm shift in conversational AI. Systems trained on large dialogue corpora demonstrated the ability to generate contextually appropriate responses to inputs not explicitly represented in training data, overcoming the principal limitation of rule-based approaches. The development of transformer-based large language models — beginning with BERT (Devlin et al., 2019) and GPT-2 (Radford et al., 2019) and scaling through GPT-3, GPT-4, and Google's PaLM and Gemini series — represented a further quantum leap in generative conversational capability.

For domain-specific tasks such as movie recommendation, the trade-off between rule-based and AI-powered approaches involves several dimensions. Rule-based systems offer predictability, auditability, zero inference latency, and zero API cost, but are limited in expressiveness and require manual maintenance of the intent pattern library and movie dataset. AI-powered systems offer open-ended conversational capability, dynamic knowledge access, nuanced intent understanding, and the ability to generate explanatory and comparative responses, but introduce API dependency, per-query costs, network latency, and potential response variability.

2.3 LLM-Based Systems (Gemini and Modern AI Chatbots)

Google's Gemini model family, introduced in December 2023, represents one of the most capable publicly accessible large language model APIs. Gemini is a natively multimodal model, trained on a diverse corpus of text, code, images, audio, and video, enabling it to reason across modalities. The Gemini Flash variant (gemini-flash-latest) is optimized for low-latency, high-throughput inference, making it particularly well-suited for interactive applications requiring rapid response generation. The Gemini API, accessible through Google AI Studio and via the

REST API, provides a developer-friendly interface for integrating generative AI capabilities into web, mobile, and backend applications.

OpenAI's ChatGPT, powered by the GPT-4 model family, has established the reference standard for conversational AI user experience. Its multi-turn dialogue capability, broad world knowledge, and ability to reason about contextual nuances make it highly effective for recommendation tasks. Anthropic's Claude demonstrates similarly strong conversational capability with a particular emphasis on helpfulness and safety. These systems, while powerful, are proprietary and subject to usage costs that scale with query volume.

Recent research on retrieval-augmented generation (RAG) systems has demonstrated that LLMs can be augmented with domain-specific knowledge retrieved at inference time, enabling them to generate recommendations informed by curated, up-to-date databases rather than relying solely on parametric knowledge encoded during training. This approach represents a promising direction for future development of CineMate, enabling integration of real-time movie databases such as The Movie Database (TMDB) or IMDb to supplement the model's parametric knowledge with current release information and user rating data.

2.4 Limitations of Existing Systems

A systematic review of existing conversational recommendation systems reveals several recurring limitations that informed the design decisions of CineMate Phase 2:

Limitation	Existing Systems Affected	CineMate Phase 2 Resolution
Static, finite knowledge base	Rule-based chatbots	Dynamic LLM knowledge via Gemini API
Brittle keyword-based intent detection	Rule-based chatbots	Natural language understanding via LLM
No genuine conversational capability	Simple rule-based bots	Full multi-turn dialogue with memory
API key exposed in client-side code	Naive API integrations	Serverless function as secure proxy

Limitation	Existing Systems Affected	CineMate Phase 2 Resolution
No structured, formatted responses	Generic LLM chatbots	Engineered prompt for structured output
No multi-session persistence	Most client-side chatbots	localStorage-based session management
High inference latency	Heavy model APIs	Gemini Flash optimized for low latency
Limited to static dataset films	Phase 1 CineMate	Full parametric LLM movie knowledge

The table above summarizes how each identified limitation of existing systems is addressed by the architectural and engineering choices made in CineMate Phase 2. The combination of LLM-powered intelligence, secure serverless API architecture, structured prompt engineering, and multi-session localStorage management collectively position CineMate Phase 2 as a comprehensive solution that overcomes the shortcomings of both its Phase 1 predecessor and comparable systems documented in the literature.

CHAPTER 3: REQUIREMENTS

3.1 Hardware Requirements

CineMate Phase 2 is designed with a minimal hardware footprint on the client side, as all heavy AI inference is offloaded to the Gemini API running on Google's cloud infrastructure. The developer workstation requirements are similarly modest, limited to the computational needs of a modern web development environment. The following table specifies the hardware requirements for development and end-user operation:

Component	Minimum Specification	Recommended Specification
Processor	Dual-core 1.8 GHz	Quad-core 2.5 GHz or higher
RAM	4 GB	8 GB or higher
Storage	500 MB free	2 GB free (for development tools)
Display	1280 x 720 resolution	1920 x 1080 Full HD or higher
Internet Connection	5 Mbps broadband (required for API)	25 Mbps or faster broadband
Browser	Chrome 90+ / Firefox 88+	Latest stable Chrome or Edge
Operating System	Windows 10 / macOS 11 / Ubuntu 20.04	Windows 11 / macOS 14 / Ubuntu 22.04

Unlike the Phase 1 system, which could operate entirely offline, CineMate Phase 2 requires a stable internet connection for its core functionality, as each recommendation query must be relayed to the Gemini API via the Vercel serverless function. The client device performs no local AI inference; its computational load is limited to rendering the UI, managing localStorage, and executing the JavaScript API communication logic.

3.2 Software Requirements

The software requirements for CineMate Phase 2 span the development toolchain, the runtime environment, and the external platform and API dependencies that underpin the system's

architecture. Each component is selected to maximize developer productivity, application security, and deployment simplicity.

3.2.1 HTML5, CSS3, and JavaScript

The frontend of CineMate Phase 2 is implemented using the standard web technology triad of HTML5, CSS3, and JavaScript (ES6+). HTML5 provides the semantic markup structure for the chat interface, including the message container, input area, and sidebar session list. CSS3 delivers the visual design — including the dark-themed colour scheme, flexbox-based layout, CSS custom properties for theme management, keyframe animations for typing indicators, and media queries for responsive adaptation across viewport sizes. JavaScript (ES6+) implements all client-side application logic: session management, localStorage interaction, DOM manipulation, API request construction and dispatch, and response rendering including markdown-to-HTML conversion for structured model output.

3.2.2 Vercel Platform

Vercel is a cloud-based deployment platform purpose-built for frontend applications and serverless functions. It provides Git-based continuous deployment, automatically building and deploying the application on each push to the connected GitHub repository. Vercel's serverless function runtime supports Node.js, enabling the implementation of the secure API proxy function (`api/chat.js`) that relays requests from the frontend to the Gemini API while keeping the API key confined to the server environment. Vercel's global edge network ensures low-latency content delivery and serverless function execution, with automatic HTTPS provisioning and a generous free tier suitable for academic projects.

3.2.3 Google Gemini API

The Google Gemini API provides programmatic access to Google's Gemini family of large language models. CineMate Phase 2 specifically targets the `gemini-flash-latest` model, which is optimized for low-latency interactive inference. API access is managed through an API key provisioned via Google AI Studio. The API accepts requests in a standardized JSON format specifying the model identifier, generation configuration parameters (temperature, maximum output tokens), a system instruction, and the conversation message array. Responses are returned as structured JSON containing the model's generated text alongside metadata about token usage and finish reason.

3.2.4 GitHub

GitHub serves as the version control and repository hosting platform for the CineMate Phase 2 codebase. The repository maintains the complete project source, including the frontend HTML, CSS, and JavaScript files, the Vercel serverless function, and the Vercel configuration file (vercel.json). The GitHub repository is connected to the Vercel deployment pipeline, enabling continuous deployment on push. This integration ensures that the production deployment always reflects the latest committed version of the codebase.

Software / Tool	Version	Role in System
HTML5	Browser-native	Chat UI structure and markup
CSS3	Browser-native	Styling, layout, dark theme, animations
JavaScript (ES6+)	Browser-native	Session logic, API calls, DOM rendering
Node.js	18.x LTS	Serverless function runtime on Vercel
Vercel CLI	Latest	Local development and deployment management
Google Gemini API	gemini-flash-latest	AI inference and recommendation generation
GitHub	Latest	Source control and CI/CD integration
VS Code	Latest	Development IDE
Git	2.40+	Version control
Google AI Studio	Web platform	API key provisioning and testing

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 Overall System Architecture

CineMate Phase 2 is built upon a four-layer architectural model that cleanly separates concerns across the UI layer, the API layer, the AI inference layer, and the client-side storage layer. This separation of concerns ensures that each architectural component can be developed, tested, and maintained independently, and that future enhancements to any individual layer do not necessitate changes to the others.

The architecture adopts a serverless deployment model, wherein no persistent backend server process is maintained. The frontend is served as a static web application from Vercel's global content delivery network, and backend logic is encapsulated in a Vercel serverless function that is instantiated on demand and terminated upon completion of each request. This model provides excellent scalability characteristics: the function scales horizontally to accommodate concurrent user sessions without requiring manual infrastructure provisioning, and incurs zero compute cost during periods of inactivity.

4.2 Architecture Flow

The end-to-end request and response flow in CineMate Phase 2 proceeds through the following sequence of steps:

- The user types a natural language movie recommendation query into the chat input field within the browser-rendered UI and submits the message by pressing Enter or clicking the Send button.
- The JavaScript frontend captures the input, appends the user message to the current session's conversation history in memory, renders the message in the chat window, displays a typing indicator, and constructs an API request payload containing the full conversation history.
- The frontend dispatches an asynchronous HTTP POST request to the relative URL `/api/chat`, which routes to the Vercel serverless function deployed alongside the static frontend assets.
- The Vercel serverless function (`api/chat.js`) receives the request, reads the `GEMINI_API_KEY` environment variable from the Vercel deployment configuration, constructs the properly formatted Gemini API request payload including the system

instruction and the conversation history, and dispatches the request to the Gemini REST API endpoint.

- The Gemini API processes the request, generates a structured recommendation response using the gemini-flash-latest model, and returns the response as a JSON object containing the generated text and usage metadata.
- The serverless function parses the Gemini response, extracts the generated text, and returns it to the frontend as a JSON response.
- The JavaScript frontend receives the response, dismisses the typing indicator, appends the assistant message to the conversation history, persists the updated session to localStorage, and renders the formatted response in the chat window, including markdown formatting such as bold text and structured sections.

4.3 Module Description

4.3.1 UI Layer

The UI Layer constitutes the entirety of the client-side presentation and interaction logic, implemented in HTML5, CSS3, and JavaScript. It is responsible for rendering the chat interface, managing user input, displaying model responses, and providing session navigation through the sidebar. The UI Layer communicates with the API Layer exclusively through asynchronous HTTP requests to the /api/chat endpoint. It maintains no direct knowledge of the Gemini API or its credentials. The layer is responsible for rendering markdown-formatted model output as rich HTML, implementing auto-scroll behaviour to keep the latest message in view, and managing the visual state of UI controls such as the typing indicator and the disabled-during-request state of the send button.

4.3.2 API Layer (Serverless Function)

The API Layer is implemented as a single Vercel serverless function located at api/chat.js. This function serves as the secure intermediary between the frontend and the Gemini API, ensuring that the API key — stored as a Vercel environment variable — is never transmitted to or accessible from the client browser. The function accepts POST requests containing the conversation message array, constructs the Gemini API request by prepending the system instruction and appending the conversation history, dispatches the request to the Gemini endpoint using the Node.js fetch API, parses the response, and returns the extracted generated text to the

calling frontend. The function includes error handling logic that returns structured error responses for cases including invalid request format, Gemini API errors, and network failures.

4.3.3 AI Layer (Gemini)

The AI Layer comprises the Google Gemini Flash model accessed via the Gemini REST API. This layer is responsible for the core intelligent reasoning: understanding the user's natural language intent, drawing upon its extensive parametric knowledge of cinema to identify appropriate recommendations, and generating a structured, well-reasoned response conforming to the prompt instructions. The gemini-flash-latest model is configured with a temperature parameter of 0.7 to balance response diversity with consistency, and a maximum output token limit of 1024 to ensure comprehensive yet concise recommendations. The system instruction embedded in each API request defines the model's persona, response format requirements, and behavioural guidelines.

4.3.4 Storage Layer (localStorage)

The Storage Layer is implemented using the browser's built-in localStorage API, which provides a synchronous key-value store with a typical capacity of 5–10 MB, sufficient for storing multiple conversation session histories. Each chat session is stored as a JSON-serialized object containing a unique session identifier, a human-readable session title derived from the first user message, the creation timestamp, and the complete conversation message array. The Storage Layer is responsible for persisting sessions across page refreshes and browser restarts, loading existing sessions on application initialization, creating new sessions on user request, and deleting sessions when the user removes them from the sidebar.

Layer	Technology	Responsibility
UI Layer	HTML5, CSS3, JavaScript	Interface rendering, user input, response display
API Layer	Vercel Serverless (Node.js)	Secure Gemini API proxy, key management
AI Layer	Google Gemini Flash API	Natural language understanding and generation
Storage Layer	Browser localStorage	Session persistence and context retention

CHAPTER 5: METHODOLOGY

5.1 Input Processing

User input in CineMate Phase 2 is processed with minimal preprocessing on the client side, as the richness and flexibility of natural language understanding is delegated entirely to the Gemini model. When the user submits a message, the frontend JavaScript trims leading and trailing whitespace from the input string, validates that the result is non-empty, and constructs a message object conforming to the Gemini API's expected format: an object with a role field set to 'user' and a parts field containing an array with a single text object. This message object is appended to the in-memory conversation history array for the current session.

The frontend does not perform tokenization, normalization, intent classification, or keyword extraction — tasks that were central to the Phase 1 rule-based engine. This architectural simplification is intentional: the Gemini model's deep language understanding capabilities render client-side NLP preprocessing redundant and potentially counterproductive, as preprocessing can inadvertently discard semantic information that the model would otherwise leverage.

5.2 API Communication Flow

The API communication flow begins when the frontend dispatches a POST request to the /api/chat endpoint with a JSON body containing the messages array representing the current conversation history. The Vercel serverless function receives this request and immediately retrieves the GEMINI_API_KEY from the process environment. It then constructs the Gemini API request body, incorporating the system instruction as the first element of the contents array with a 'model' role designation, followed by the conversation message history provided by the frontend.

The serverless function dispatches a POST request to the Gemini REST API endpoint (<https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent>), passing the API key as a query parameter and the constructed request body as the JSON payload. Upon receiving the Gemini response, the function extracts the generated text from the deeply nested response structure (`response.candidates[0].content.parts[0].text`), constructs a simple JSON response object containing the reply field, and returns it to the frontend with a 200 HTTP status code. Error conditions at any stage result in an appropriate error status code and a JSON error message being returned to the frontend, which displays a user-friendly error notification in the chat interface.

5.3 Prompt Engineering Strategy

The system instruction embedded in each Gemini API request is the central mechanism through which CineMate Phase 2 shapes the model's behaviour, persona, and response format. The system instruction performs several critical functions: it establishes the model's identity as CineMate, an expert movie recommendation assistant; it instructs the model to always structure its responses in a specific four-section format; it provides formatting guidance for each section; and it instructs the model to draw upon its comprehensive cinematic knowledge to provide thoughtful, well-reasoned recommendations.

The four-section response format mandated by the system instruction comprises: an Overview section that acknowledges the user's query and provides contextual framing; a Movies section that lists the recommended films with titles and brief synopses; a Why section that explains the reasoning behind each recommendation in relation to the user's expressed preferences; and an Examples section that highlights specific scenes, moments, or qualities of each recommended film that exemplify why it suits the user's request. This structured format ensures that model responses are consistently organized, informative, and engaging, regardless of the specific nature of the user's query.

5.4 Context Memory Handling

Conversational context memory in CineMate Phase 2 is implemented through a stateless, history-passing architecture. Unlike server-side session management approaches that maintain conversation state in a database or in-memory cache, CineMate Phase 2 transmits the complete conversation history with every API request. The frontend maintains the current session's message array in JavaScript memory, updated with each user message and model response. When a new user message is submitted, the entire accumulated message array — containing all prior user and assistant turns — is included in the POST request body sent to the serverless function, which passes it verbatim to the Gemini API.

This approach exploits the Gemini model's context window — its ability to attend to and reason over the full conversation history provided in the input — to produce contextually coherent responses that appropriately reference and build upon prior turns. From the user's perspective, the chatbot appears to 'remember' previous messages within a session and can meaningfully respond to follow-up queries such as 'what about something similar but from the 1990s?' or 'can you suggest the director's other films?' without requiring the user to restate context. The

localStorage mechanism ensures that session histories persist across browser sessions, enabling users to resume prior conversations.

5.5 Response Structuring (Overview, Movies, Why, Examples)

The structured four-section response format is enforced through explicit instructions in the system prompt. The model is instructed to format its response using markdown conventions — bold section headers denoted with double asterisks, dashed list items for movie titles, and appropriately indented sub-content — which the frontend JavaScript renders as formatted HTML using a lightweight markdown parser.

Section	Purpose	Typical Content
Overview	Contextual framing and query acknowledgement	2-3 sentences interpreting the user's request and setting expectations for the recommendations
Movies	Core recommendation list	3-5 film titles with year and brief one-sentence synopsis each
Why	Reasoning and justification	Explanation of how each recommendation matches the user's expressed preferences, mood, or context
Examples	Specific cinematic highlights	Memorable scenes, visual qualities, tonal aspects, or narrative moments that exemplify the recommendation's suitability

CHAPTER 6: IMPLEMENTATION AND DEPLOYMENT

6.1 File Structure Explanation

The CineMate Phase 2 project follows a clean, minimal directory structure designed for straightforward development and seamless Vercel deployment. The root directory contains the primary HTML file (`index.html`), which serves as the single-page application shell. A separate CSS file (`style.css`) encapsulates all visual styling, and the JavaScript application logic is housed in `script.js`. The `api/` subdirectory contains the Vercel serverless function (`chat.js`) that proxies requests to the Gemini API. A `vercel.json` configuration file at the root level defines routing rules and environment variable references for the Vercel deployment.

File / Directory	Type	Description
<code>index.html</code>	HTML	Main application shell: chat UI, sidebar, input area
<code>style.css</code>	CSS	Full application styling: dark theme, layout, animations
<code>script.js</code>	JavaScript	Session management, API calls, DOM rendering, <code>localStorage</code>
<code>api/chat.js</code>	Node.js	Vercel serverless function: Gemini API secure proxy
<code>vercel.json</code>	JSON	Vercel deployment configuration and routing rules
<code>.env.local</code>	Config	Local development environment variables (not committed)
<code>README.md</code>	Markdown	Project documentation and setup instructions

6.2 Frontend Implementation

6.2.1 HTML Structure (`index.html`)

The `index.html` file defines the structural skeleton of the CineMate interface. The layout is divided into two primary sections: a sidebar containing the session list, a new-chat button, and the application title; and a main content area comprising the chat message container, a typing indicator element, and the message input area with a send button. HTML5 semantic elements are used throughout to ensure accessibility. All interactive elements carry appropriate ARIA attributes. The document links the external `style.css` for styling and the `script.js` at the body's end to ensure the DOM is fully parsed before JavaScript execution begins.

6.2.2 CSS Design (style.css)

The stylesheet implements a comprehensive dark-themed design using CSS custom properties defined on the `:root` selector for centralized theme management. The colour palette uses deep navy and charcoal background tones, white and light-grey text, and a blue accent colour for interactive elements and the CineMate brand identity. Flexbox is used for the primary layout — placing the sidebar and main chat area in a horizontal row — and for the internal layout of the message input area. CSS keyframe animations power the three-dot typing indicator that provides visual feedback during API request processing. Media queries provide responsive adaptation for narrow viewport widths, collapsing the sidebar and adjusting font sizes appropriately.

6.2.3 JavaScript Application Logic (script.js)

The `script.js` file implements the complete client-side application logic. On initialization, the script loads existing sessions from `localStorage`, renders the session list in the sidebar, and restores the most recently active session or creates a new default session if none exists. The `sendMessage()` function constitutes the core application workflow: it captures and validates the user input, constructs and appends the user message object to the session history, calls the `renderMessage()` function to display the message in the chat window, displays the typing indicator, and dispatches the API request via the `callGeminiAPI()` function. Upon receiving the API response, `sendMessage()` appends the assistant message to the session history, persists the updated session to `localStorage` via the `saveSession()` function, and calls `renderMessage()` to display the formatted response.

The `renderMessage()` function converts markdown-formatted model output to HTML by processing bold text (double asterisk patterns), unordered list items (dash-prefixed lines), and line breaks, producing a structured and visually readable presentation of the four-section recommendation response. Session management functions (`createNewSession`, `switchSession`, `deleteSession`, `saveSession`, `loadSessions`) implement the complete session lifecycle, persisting all session data as JSON-serialized objects in `localStorage` under unique session identifier keys.

6.3 Serverless Function Implementation (api/chat.js)

The `api/chat.js` serverless function is implemented as a standard Vercel API route using the Node.js runtime. The function exports a default async handler that accepts the HTTP request and response objects. It first validates that the request method is POST, returning a 405 Method Not Allowed response for other methods. It then destructures the messages array from the parsed

request body and validates its presence and format, returning a 400 Bad Request response if the validation fails.

The function retrieves the Gemini API key from `process.env.GEMINI_API_KEY`, constructs the Gemini request body by prepending a system instruction as a model-role turn and appending the user-provided message history, and dispatches the request to the Gemini REST API endpoint using the built-in `fetch` function available in Node.js 18+. Upon receiving the Gemini response, the function checks the HTTP status and parses the JSON body, extracting the generated text from the nested response structure. The extracted text is returned to the client as a JSON object with a `reply` field. A comprehensive try-catch block ensures that all uncaught exceptions produce a structured 500 Internal Server Error response rather than an unhandled promise rejection.

6.4 Gemini API Integration using gemini-flash-latest

The Gemini API integration is configured to use the `gemini-flash-latest` model, specified in the model path segment of the API endpoint URL. The generation configuration object passed in the request body specifies a temperature of 0.7, balancing creative variation in recommendations with consistent adherence to the structured response format. The `maxOutputTokens` parameter is set to 1024, providing sufficient capacity for a complete four-section recommendation response while preventing excessively long outputs that could degrade the user experience. The system instruction is embedded as the first element of the `contents` array with a role of `'model'`, a convention that the Gemini API uses to establish the model's persona and behavioural guidelines prior to the conversation history.

6.5 Deployment on Vercel

Deployment of CineMate Phase 2 on Vercel involves connecting the GitHub repository to a Vercel project via the Vercel dashboard. On each push to the main branch, Vercel's build pipeline automatically detects the project structure, identifies the `api/` directory as containing serverless functions, serves the root-level HTML, CSS, and JavaScript files as static assets from its global CDN, and deploys the serverless function to Vercel's edge network. The `GEMINI_API_KEY` is configured as an environment variable in the Vercel project settings, accessible to the serverless function at runtime via `process.env` but never exposed in the static frontend assets or client-side JavaScript.

The `vercel.json` configuration file specifies route rewrites to ensure that requests to `/api/chat` are handled by the serverless function and that all other requests are served the `index.html` file,

enabling client-side navigation. Vercel automatically provisions an HTTPS certificate for the deployment's subdomain, ensuring all communication between the client browser, the Vercel CDN, the serverless function, and the Gemini API is encrypted in transit.

CHAPTER 7: EXPERIMENTAL RESULTS

7.1 Testing with Different Inputs

CineMate Phase 2 was subjected to a structured evaluation protocol comprising twenty-five distinct test queries spanning a range of input types and complexity levels. The test categories included simple genre queries, mood-based queries, context-specific queries, compound multi-criteria queries, follow-up queries testing context retention, edge-case queries with ambiguous or unusual phrasing, and comparative queries requesting evaluation of multiple films or directors. Representative test cases and their corresponding system responses are summarized in the following table:

Query Type	Sample Query	Response Quality	Sections Present
Simple Genre	Recommend some sci-fi movies	High — 5 relevant titles with rationale	All 4 sections
Mood-Based	I want something uplifting and feel-good	High — tone-appropriate selections	All 4 sections
Context-Specific	A film for a quiet rainy afternoon alone	High — atmospheric matches	All 4 sections
Compound	Dark thriller with psychological elements from the 2010s	High — precise era and tone match	All 4 sections
Follow-up	What about something similar but lighter?	High — correct context inference	All 4 sections
Director Query	Recommend films by Christopher Nolan	High — filmography knowledge	All 4 sections
Ambiguous	Something interesting	Moderate — broad but reasonable picks	All 4 sections
Non-English	Best Korean films of the last decade	High — accurate regional knowledge	All 4 sections

7.2 Output Accuracy and Relevance

Output accuracy and relevance were evaluated using a three-point rubric applied by the evaluator to each test case response: a score of 3 indicated that all recommended films were fully relevant to the query intent; a score of 2 indicated that the majority of recommendations were relevant with minor mismatches; and a score of 1 indicated that fewer than half of the recommendations were relevant or that the response significantly misinterpreted the query. Across the twenty-five test queries, the system achieved a mean relevance score of 2.84 out of 3.0, indicating a very high rate of accurate, intent-appropriate recommendation generation. Scores of 3 were assigned to twenty-one of the twenty-five queries, with the four remaining queries receiving scores of 2 — all four were instances of highly ambiguous input where multiple reasonable interpretations existed.

The structured four-section response format was produced consistently across all twenty-five queries, demonstrating that the prompt engineering strategy successfully enforces the desired output structure. Minor formatting variations — such as occasional use of numbered lists instead of dashed lists in the Movies section — were observed in three queries but did not significantly impact readability or information quality.

7.3 Context Handling Verification

Context retention was evaluated through a series of five multi-turn conversation scenarios, each comprising between three and six turns. In each scenario, the evaluator assessed whether the model's responses to follow-up queries correctly incorporated and referenced the context established in prior turns without requiring the user to restate that context explicitly.

Scenario	Turns	Context Elements	Retention Accuracy
Genre refinement chain	4	Genre → Sub-genre → Era → Language	100%
Mood progression	3	Initial mood → Intensity preference → Runtime	100%
Director exploration	5	Director → Film style → Actor preference	100%

Scenario	Turns	Context Elements	Retention Accuracy
Negative feedback handling	3	Disliked genre → Alternative suggestions	100%
Compound multi-session test	6	Cross-turn genre, era, and tone memory	100%

Context retention accuracy was 100% across all evaluated scenarios, confirming that the history-passing architecture reliably maintains conversational coherence. The Gemini model consistently demonstrated the ability to interpret follow-up queries such as 'something similar but from the 1980s' or 'what about horror instead?' in light of the prior conversation context, producing responses that accurately reflected the accumulated preference history.

7.4 Performance Evaluation

Response latency was measured across fifty API requests dispatched under standard broadband conditions (approximately 50 Mbps download), spanning a range of query complexities and response lengths. Latency was measured from the moment the POST request was dispatched by the frontend JavaScript to the moment the response JSON was received and parsed.

Metric	Value
Minimum Response Latency	0.82 seconds
Maximum Response Latency	3.41 seconds
Mean Response Latency	1.74 seconds
Median Response Latency	1.61 seconds
Standard Deviation	0.48 seconds
Requests with Latency < 2s	78% of requests
Requests with Latency > 3s	6% of requests

Metric	Value
Serverless Function Cold Start (first request)	~1.2 seconds additional

The observed latency profile is consistent with the performance characteristics of the Gemini Flash model, which is specifically optimized for interactive, low-latency inference. The elevated latency observed in the first request of each session reflects the serverless function cold start — a brief initialization period that occurs when the function container is instantiated for the first time or after a period of inactivity. Subsequent requests within the same session exhibit consistently lower latency as the container remains warm.

CHAPTER 8: DISCUSSION

8.1 Comparison with Phase 1 Chatbot

A systematic comparison of CineMate Phase 1 and Phase 2 across key functional and architectural dimensions reveals the significant scope of advancement achieved in the second phase. The Phase 1 system, while technically accomplished within its constrained design parameters, was fundamentally bounded by its static knowledge base and rule-based intent detection logic. The Phase 2 system addresses each of the identified Phase 1 limitations through targeted architectural decisions, as summarized in the following comparative table:

Dimension	Phase 1 (Rule-Based)	Phase 2 (AI-Powered)
Knowledge Source	Static JavaScript dataset (~50 films)	Gemini parametric knowledge (thousands of films)
Intent Understanding	Keyword matching, brittle	Natural language understanding, robust
Response Generation	Template-based, deterministic	LLM-generated, dynamic and explanatory
Conversational Ability	Single-turn only	Full multi-turn with context memory
Backend Architecture	None (fully client-side)	Vercel serverless function (secure proxy)
API Security	N/A	API key in server environment only
Response Structure	Hardcoded template format	Prompt-enforced four-section structure
Internet Required	No (fully offline)	Yes (API required)
Recommendation Variety	Limited to dataset films	Unlimited via LLM knowledge

Dimension	Phase 1 (Rule-Based)	Phase 2 (AI-Powered)
Deployment	Static file serving / GitHub Pages	Vercel CDN + Serverless Functions

8.2 Improvements Achieved

The most impactful improvement achieved in Phase 2 is the transition from static, dataset-bounded recommendations to dynamically generated, knowledge-rich recommendations powered by the Gemini language model. In Phase 1, a user asking about a film released after the dataset was last updated would receive no relevant recommendation; in Phase 2, the model can draw upon its training knowledge to recommend films from any era, genre, or cultural context, including nuanced categories such as 'Scandinavian neo-noir' or 'Korean social commentary films' that would have been intractable for the Phase 1 keyword engine.

The implementation of genuine multi-turn conversational memory represents a qualitative improvement in user experience. Phase 1 users were required to fully specify their preferences in each query; Phase 2 users can engage in a natural dialogue, progressively refining their preferences across multiple turns without repetition. The structured four-section response format, enforced through prompt engineering rather than hardcoded templates, produces responses that are simultaneously more informative and more flexible than Phase 1's template-driven output, as the model can adapt the content and depth of each section to the specific nature of the query.

8.3 Observations

Several noteworthy observations emerged from the evaluation of CineMate Phase 2. First, the model demonstrates strong awareness of cultural, regional, and temporal context in its recommendations, accurately distinguishing between stylistic conventions of different cinematic traditions (Hollywood commercial cinema versus European art cinema, for instance) and correctly interpreting era-specific queries. Second, the model occasionally exhibits a tendency toward recommending well-known, critically acclaimed films even when the user's phrasing might suggest an appetite for more obscure or cult selections; a future enhancement to the system prompt could explicitly instruct the model to include at least one lesser-known recommendation per response.

Third, response quality exhibits some sensitivity to the generality of the user's input: highly specific queries (with genre, era, tone, and language specified) consistently produced the most

targeted and relevant recommendations, while highly vague queries (such as 'something good') produced competent but less precisely tailored responses. This observation suggests that incorporating a conversational elicitation step — where the model asks clarifying questions before generating recommendations for ambiguous queries — could further improve user satisfaction for underspecified inputs.

CHAPTER 9: ADVANTAGES AND LIMITATIONS

9.1 Advantages

9.1.1 AI-Powered Dynamic Responses

The integration of the Gemini Flash language model endows CineMate Phase 2 with the ability to generate recommendations that are dynamically tailored to each user's specific, nuanced query — a capability fundamentally beyond the reach of rule-based systems. The model draws upon deep parametric knowledge of thousands of films across all genres, eras, languages, and cultural traditions, enabling it to respond accurately to highly specific requests such as 'cerebral science fiction films that explore the nature of consciousness, similar in tone to Arrival' with a level of accuracy, depth, and explanatory richness that no manually curated dataset could match.

9.1.2 Personalized Recommendations

The conversational memory architecture enables a form of within-session personalization that progressively refines recommendations in response to user feedback expressed through natural dialogue. As the conversation develops, the model accumulates contextual signals about the user's preferences, dislikes, and situational viewing context, producing recommendations that are increasingly tailored to the individual over the course of the session. This contrasts sharply with the stateless, context-blind operation of the Phase 1 system and represents a meaningful step toward genuinely personalized movie discovery.

9.1.3 Scalable Serverless Architecture

The serverless deployment architecture on Vercel provides inherent scalability without requiring manual infrastructure management. As user demand increases — whether from tens of concurrent users to thousands — the Vercel platform automatically provisions additional serverless function instances to handle the load, with no code changes or infrastructure intervention required. This scalability is achieved at minimal cost, as serverless pricing models charge only for actual compute time consumed by function executions rather than for persistent server capacity.

9.1.4 Secure API Key Management

The implementation of the serverless function as a secure API proxy ensures that the Gemini API key is never exposed in client-side code, preventing unauthorized use by third parties who might extract the key from browser developer tools or network inspection. This security

architecture conforms to industry best practices for web application development and ensures that the project can be deployed as a production application without API key compromise risk.

9.2 Limitations

9.2.1 Internet and API Dependency

CineMate Phase 2's core functionality is entirely dependent on internet connectivity and the availability of the Gemini API. In the absence of network access, or during periods of Gemini API downtime or degraded service, the chatbot is completely non-functional. This represents a significant regression from Phase 1, which operated entirely offline. Users in environments with unreliable internet connectivity, corporate network restrictions that block API traffic, or during Gemini API maintenance windows will be unable to use the system.

9.2.2 Response Variability

The stochastic nature of LLM inference — controlled by the temperature parameter — means that identical queries submitted at different times may produce different recommendations. While this variation is often beneficial, producing diverse and non-repetitive responses across sessions, it can occasionally result in recommendations of inconsistent quality or relevance for edge-case queries. Unlike the Phase 1 system, whose deterministic scoring algorithm produced identical outputs for identical inputs, Phase 2's response quality cannot be guaranteed to a fixed standard for all possible inputs.

9.2.3 API Usage Limits and Costs

The Gemini API is subject to usage quotas and rate limits that may constrain the system's availability under high concurrent usage scenarios. Exceeding the free tier usage quota would require transitioning to a paid API plan, introducing ongoing operational costs. For academic and personal deployment scenarios, the free tier is typically sufficient, but production deployment at significant scale would necessitate careful cost management and quota monitoring.

Factor	Advantage / Limitation	Severity / Impact
AI-powered recommendations	Advantage	High — core differentiator
Conversational personalization	Advantage	High — superior UX

Factor	Advantage / Limitation	Severity / Impact
Serverless scalability	Advantage	High — production-ready
Secure API proxy	Advantage	High — security compliance
Internet dependency	Limitation	Medium — requires connectivity
Response variability	Limitation	Low — manageable via prompt tuning
API rate limits	Limitation	Low — within academic scope

CHAPTER 10: CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

CineMate Phase 2 represents a comprehensive and architecturally significant evolution from its rule-based predecessor, demonstrating the transformative impact that large language model integration can have on the capability, flexibility, and user experience of domain-specific conversational applications. By replacing the static movie dataset and keyword-matching intent engine of Phase 1 with the dynamic, knowledge-rich inference capabilities of the Google Gemini Flash model, Phase 2 delivers a recommendation experience that is qualitatively superior across every evaluated dimension: recommendation breadth and accuracy, conversational naturalness, response informativeness, and architectural scalability.

The implementation of a secure serverless API proxy on the Vercel platform ensures that the system adheres to production security standards while maintaining the simplicity and accessibility of a static web application for end users. The structured four-section response format, enforced through careful prompt engineering, produces consistently organized and informative recommendations that exceed the expressiveness achievable through hardcoded template systems. The history-passing conversational memory architecture enables genuine multi-turn dialogue, allowing users to progressively refine their preferences through natural conversation rather than isolated, context-free queries.

Experimental evaluation confirms that the system achieves high recommendation relevance (mean score 2.84/3.0), perfect context retention across multi-turn scenarios, and acceptable response latency (mean 1.74 seconds) under standard network conditions. These results validate the design choices made throughout the development of CineMate Phase 2 and establish it as a production-capable, academically rigorous demonstration of AI-powered conversational recommendation system design.

10.2 Future Scope

10.2.1 Voice-Based Interaction

A natural next evolution of the CineMate interface is the integration of voice input and output capabilities, transforming the text-based chat interaction into a hands-free, voice-driven experience. The Web Speech API, natively supported in modern browsers, provides both speech-to-text transcription for voice input and text-to-speech synthesis for voice output. Integration of these APIs would enable users to speak their movie preference queries aloud and receive spoken

recommendation responses, significantly enhancing accessibility for users with visual impairments or limited keyboard access and creating a more natural, ambient interaction paradigm analogous to voice assistants such as Amazon Alexa or Google Assistant.

10.2.2 Multi-Language Support

The Gemini model's native multilingual capability makes multi-language support a readily achievable enhancement. Implementing automatic language detection on the user's input — using the browser's `navigator.language` API or a dedicated language identification library — and configuring the system prompt to instruct the model to respond in the detected language would enable CineMate to serve users in their native language without interface modification. This enhancement would significantly broaden the system's global accessibility, enabling users across linguistic communities to discover cinema in languages and regional traditions most relevant to their cultural context.

10.2.3 User Preference Learning

Cross-session personalization, enabled by learning and persisting user preference signals over multiple sessions, represents a compelling long-term enhancement. A preference profile stored in `localStorage` could accumulate signals from each session — favourite genres, preferred eras, disliked content categories, previously recommended films — and incorporate these signals into the system prompt as a personalized context block, enabling the model to generate recommendations that reflect the user's evolving taste profile across sessions. Integration with the TMDb or IMDb APIs would further enhance this capability by enabling the system to mark recommended films as watched and exclude them from future sessions, creating a persistent, growing watchlist-aware recommendation experience.

10.2.4 Retrieval-Augmented Generation

Integration of a retrieval-augmented generation (RAG) pipeline would enable CineMate to supplement the Gemini model's parametric knowledge with current, structured data retrieved at query time from movie databases such as The Movie Database (TMDb). This enhancement would address the model's knowledge cutoff limitation, enabling accurate recommendations of films released after the model's training data cutoff date and providing the model with current user ratings, box office data, and streaming platform availability information to further enrich its recommendation reasoning.

CHAPTER 11: REFERENCES

1. Google. (2024). *Gemini API Documentation*. Google AI for Developers. Available at: <https://ai.google.dev/docs>
2. Google. (2024). *Gemini API Reference: generateContent*. Available at: <https://ai.google.dev/api/generate-content>
3. Vercel Inc. (2024). *Vercel Serverless Functions Documentation*. Available at: <https://vercel.com/docs/functions>
4. Mozilla Developer Network. (2024). *Fetch API*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
5. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30.
6. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., et al. (2020). *Language models are few-shot learners*. Advances in Neural Information Processing Systems, 33, 1877–1901.
7. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. Proceedings of NAACL-HLT 2019.
8. Team, G., Anil, R., Borgeaud, S., et al. (2023). *Gemini: A family of highly capable multimodal models*. arXiv preprint arXiv:2312.11805.
9. Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix factorization techniques for recommender systems*. Computer, 42(8), 30–37.
10. Gomez-Uribe, C. A., & Hunt, N. (2016). *The Netflix recommender system: Algorithms, business value, and innovation*. ACM Transactions on Management Information Systems.